

what does a computer actually do?

digitalisierung und programmierung · prof. dr. nicolas meseth

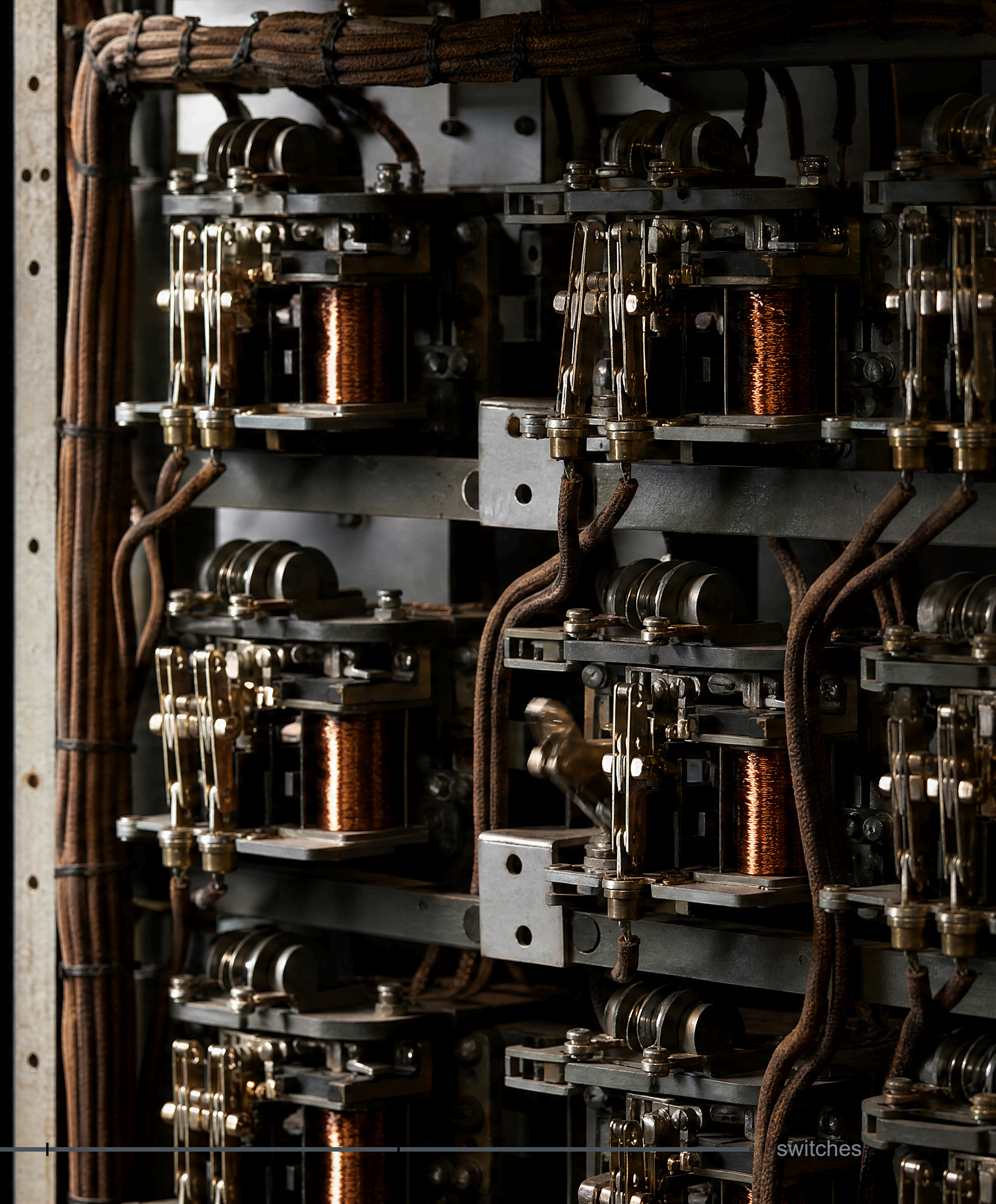
- 1 switches
- 2 storing
- 3 adding
- 4 a photo, pixel by pixel
- 5 one gate, two jobs

part 1

switches

what can you build
out of switches?

image: ai-generated (gpt-image-2)



switches

three things a computer does with bits

store them

three things a computer does with bits

store them

move them

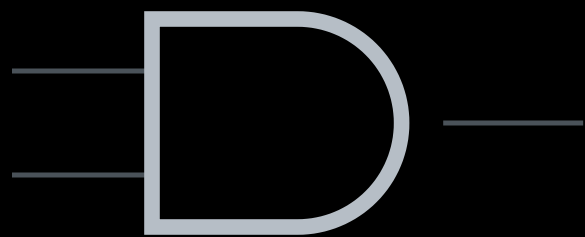
three things a computer does with bits

store them

move them

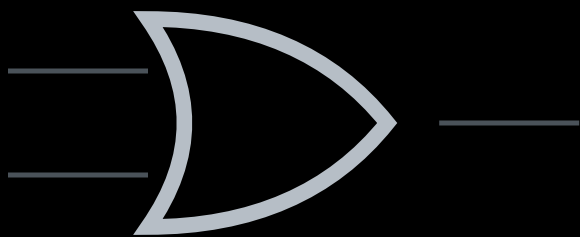
change them by rules

the toolbox: and, or, xor



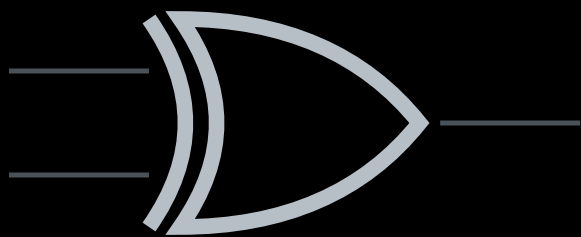
and

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1



or

a	b	out
0	0	0
0	1	1
1	0	1
1	1	1

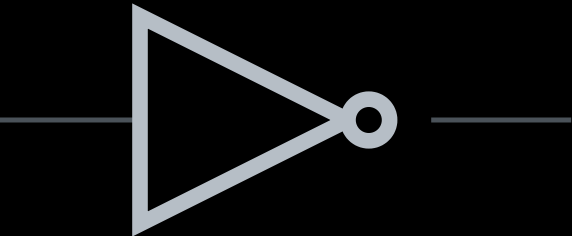


xor

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

two bits in, one bit out. a rule, nothing else.

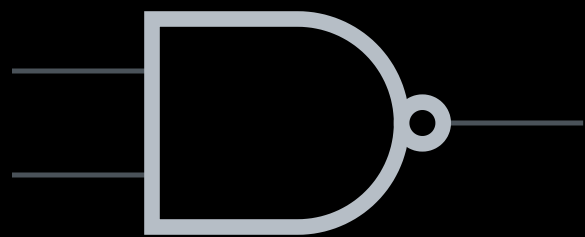
and one that flips: not



not

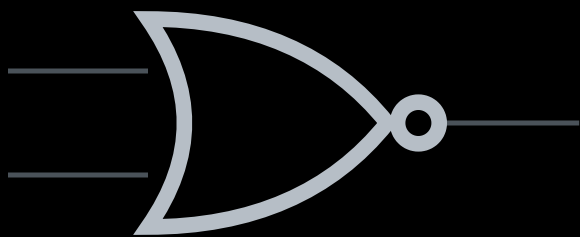
a	out
0	1
1	0

with a not behind them: nand, nor, xnor



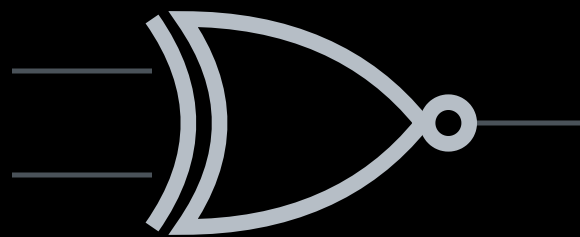
nand

a	b	out
0	0	1
0	1	1
1	0	1
1	1	0



nor

a	b	out
0	0	1
0	1	0
1	0	0
1	1	0



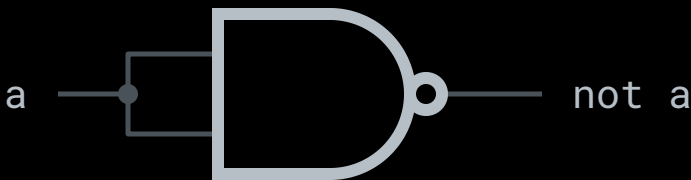
xnor

a	b	out
0	0	1
0	1	0
1	0	0
1	1	1

the small circle at the output means: and then flip it.

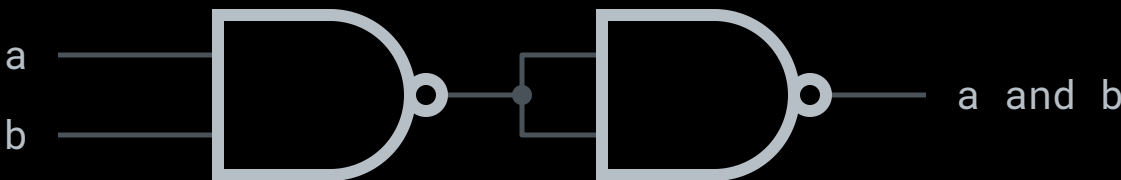
one gate is enough: nand

not



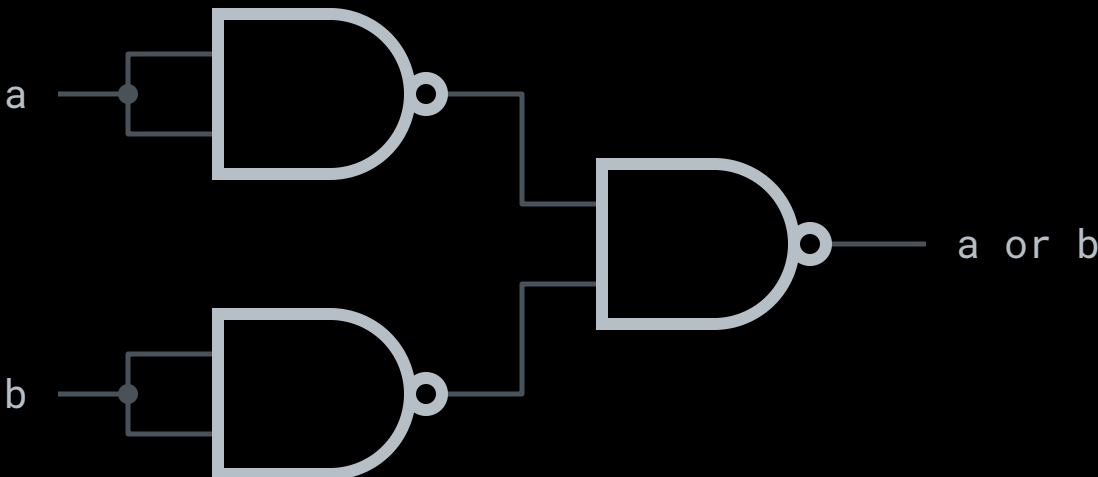
the same bit into both inputs

and



nand, then flipped again

or



both flipped first, then nand

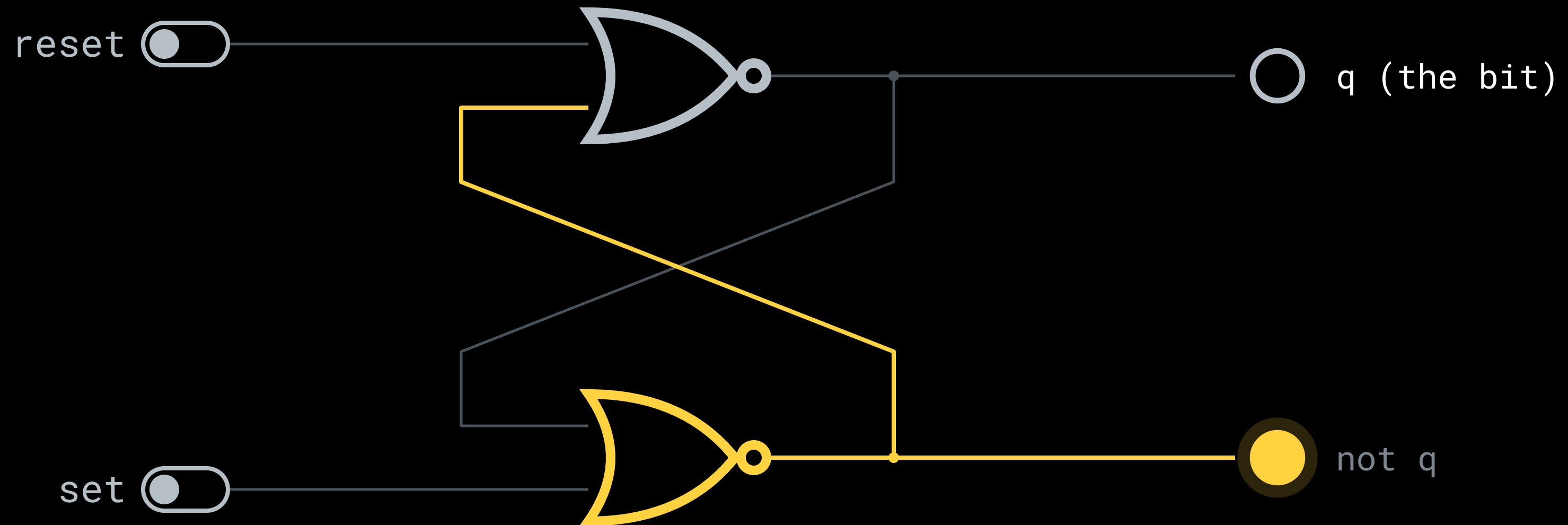
every chip is mostly nand. everything else is built from it.

no gate understands anything.
it just switches.

part 2

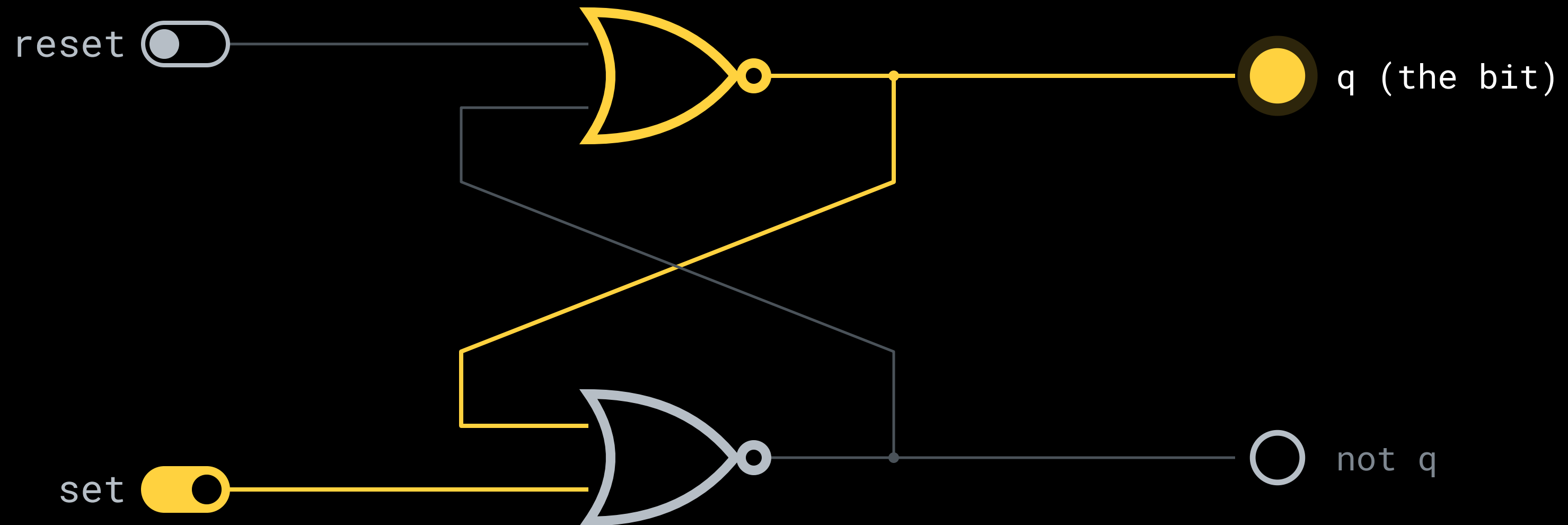
storing

two nor gates, wired into each other



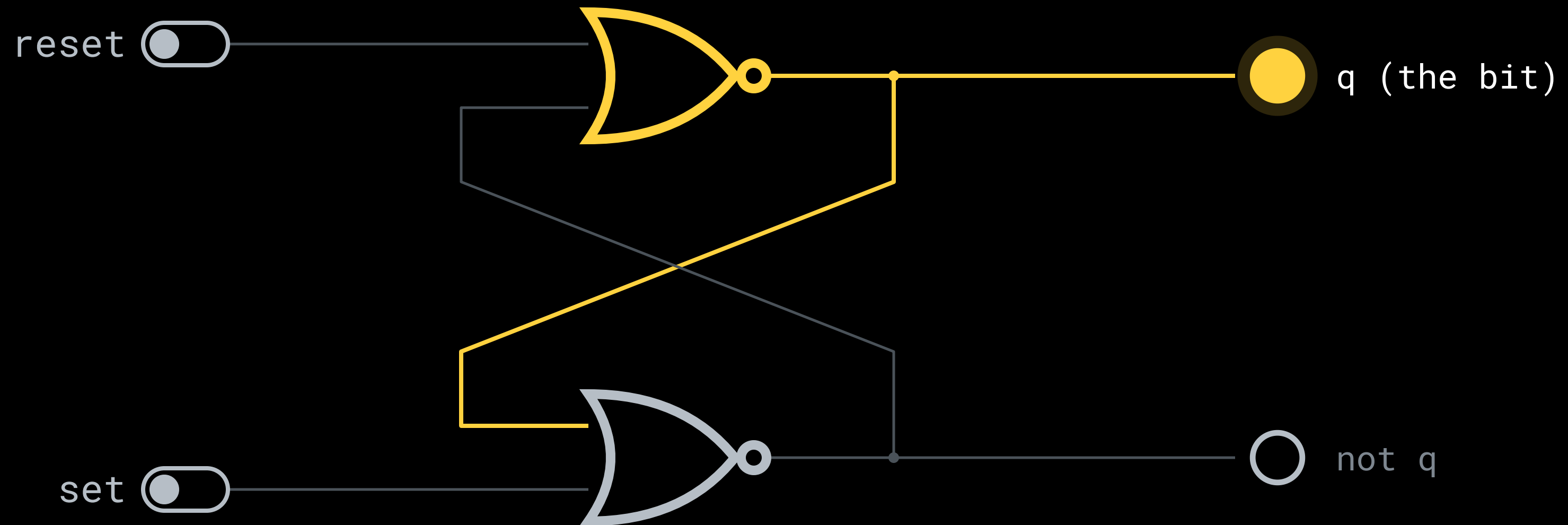
set = 0, reset = 0: nothing happens yet. q is 0.

two nor gates, wired into each other



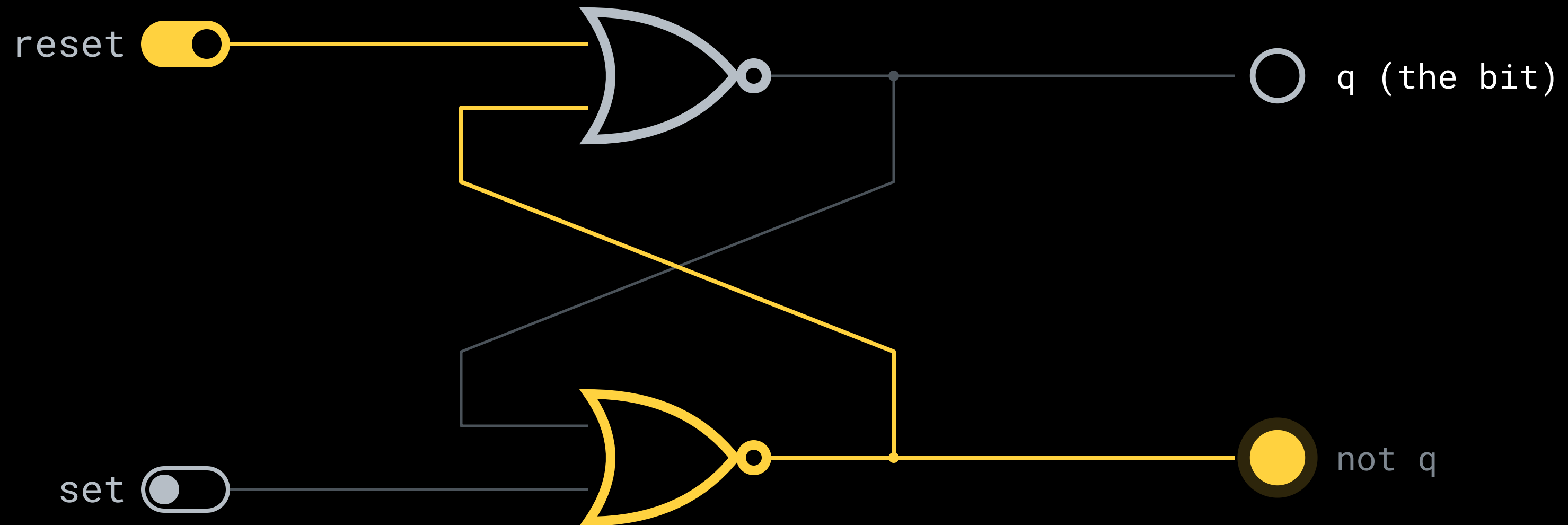
set = 1: the lower nor gives 0, the upper gets two zeros and gives 1. $q = 1$.

two nor gates, wired into each other



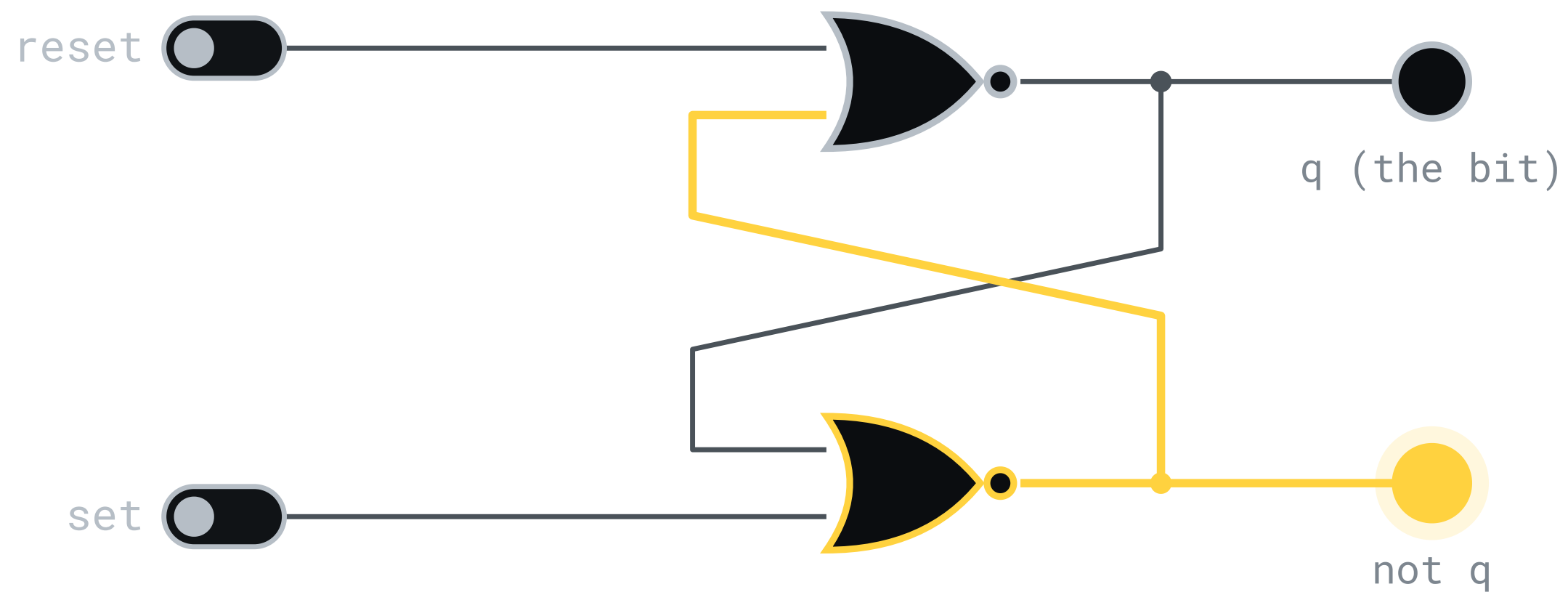
set back to 0: q stays 1. the loop holds it. the circuit remembers.

two nor gates, wired into each other



reset = 1: q drops to 0 and stays there. set, hold, clear.

try it: the flip-flop

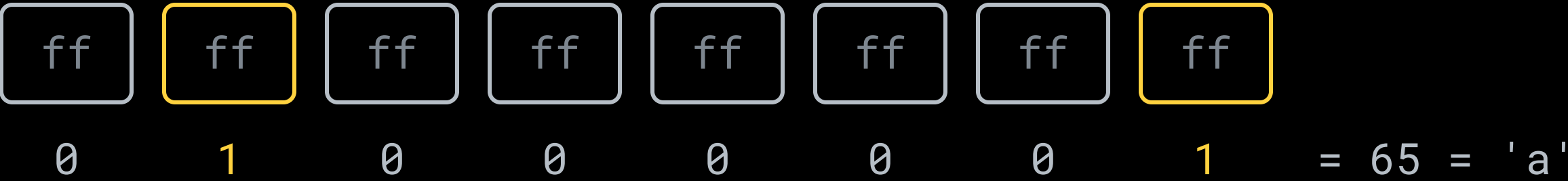


both off $\rightarrow$ q holds 0

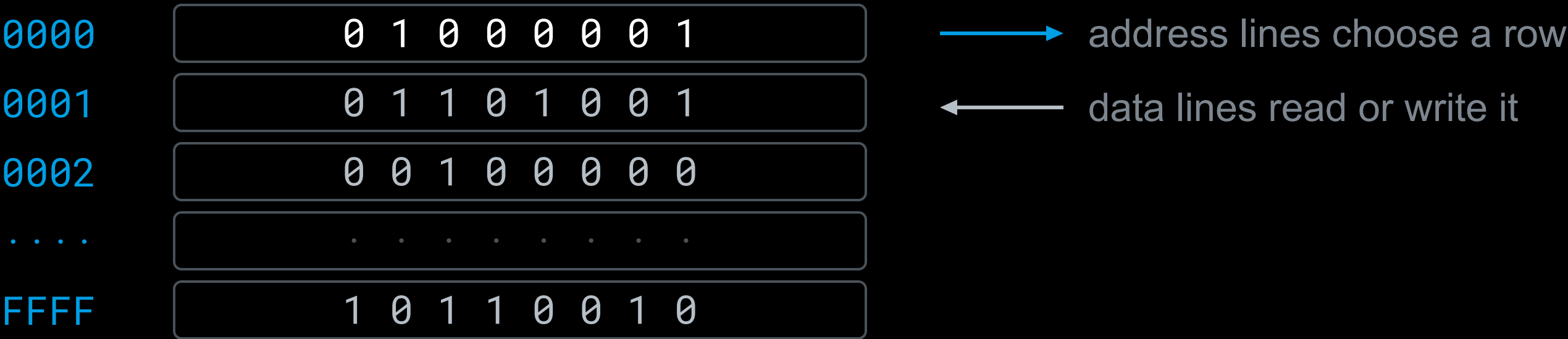
no clock, no memory chip: the bit lives in the loop between the two gates, as long as the power stays on. this is one bit of ram. see the register for eight of them.

from one bit to memory

a register: eight flip-flops, one byte



RAM: millions of rows, each with an address



power off, and the loops collapse: RAM forgets.

excursion: what 32-bit and 64-bit mean

16 address lines choose one of 65,536 rows: 64 KB

1111 1111 1111 1111

the highest address a home computer from 1982 could name

every extra address line doubles the memory a computer can name.

excursion: what 32-bit and 64-bit mean

16 address lines choose one of **65,536** rows: 64 KB

1111 1111 1111 1111

the highest address a home computer from 1982 could name

32 address lines choose one of **4,294,967,296** rows: 4 GB

1111 1111 1111 1111 1111 1111 1111 1111

the wall of 32-bit windows: a fifth gigabyte of RAM has no address

every extra address line doubles the memory a computer can name.

excursion: what 32-bit and 64-bit mean

16 address lines choose one of **65,536** rows: 64 KB

1111 1111 1111 1111

the highest address a home computer from 1982 could name

32 address lines choose one of **4,294,967,296** rows: 4 GB

1111 1111 1111 1111 1111 1111 1111 1111

the wall of 32-bit windows: a fifth gigabyte of RAM has no address

64 address lines choose one of **18,446,744,073,709,551,616** rows: 16 EB

1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111

16 billion gigabytes: your laptop today, with room for a long time

every extra address line doubles the memory a computer can name.

part 3

adding

1 + 1 in binary

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

1 + 1 in binary

$$0 + 0 = 00$$

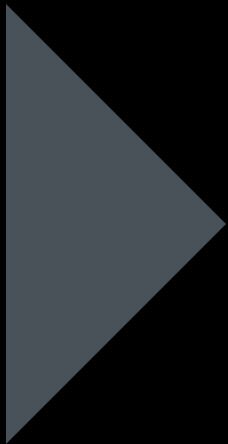
$$0 + 1 = 01$$

$$1 + 0 = 01$$

$$1 + 1 = 10$$

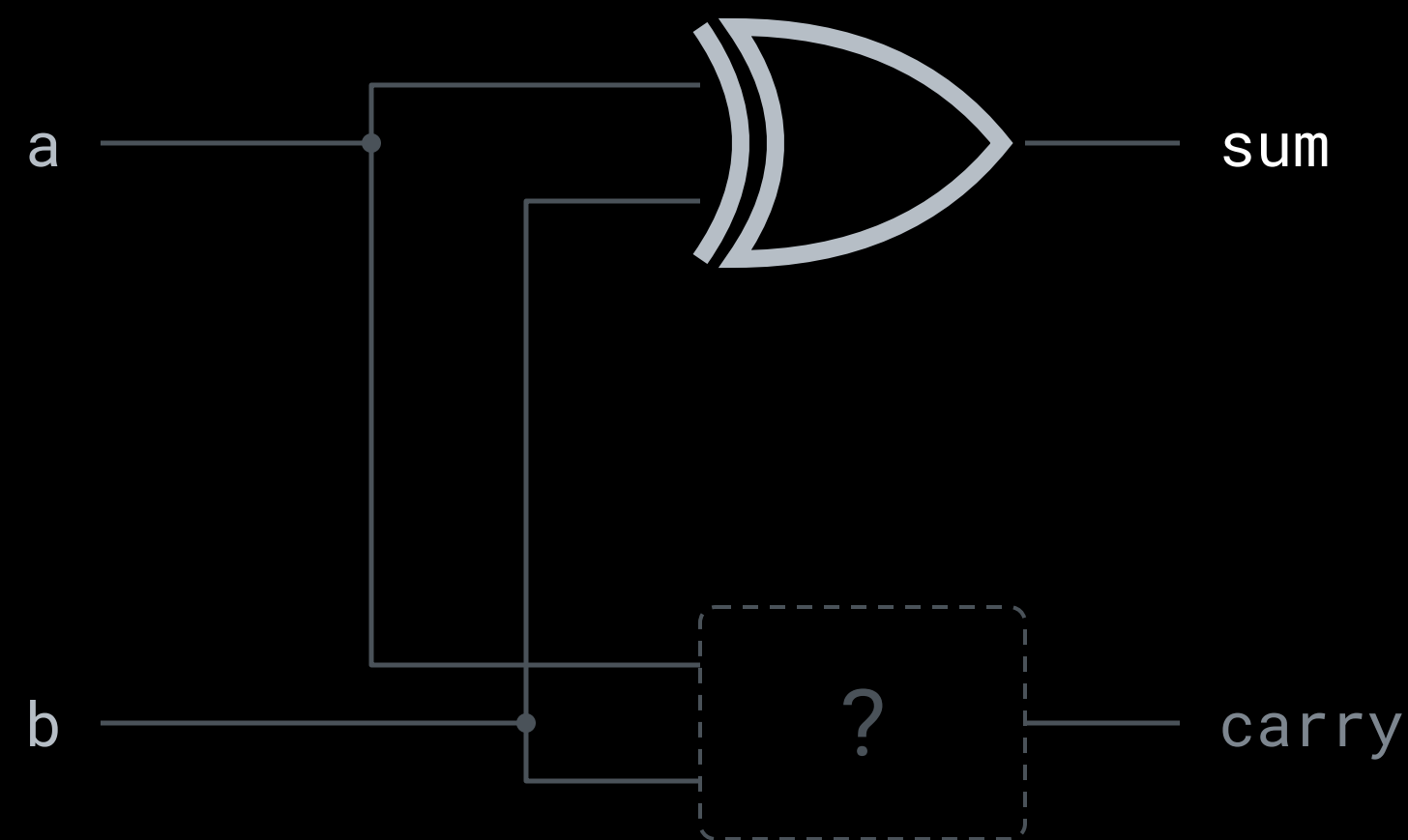
1 + 1 in binary

0 + 0 = 00
0 + 1 = 01
1 + 0 = 01
1 + 1 = 10



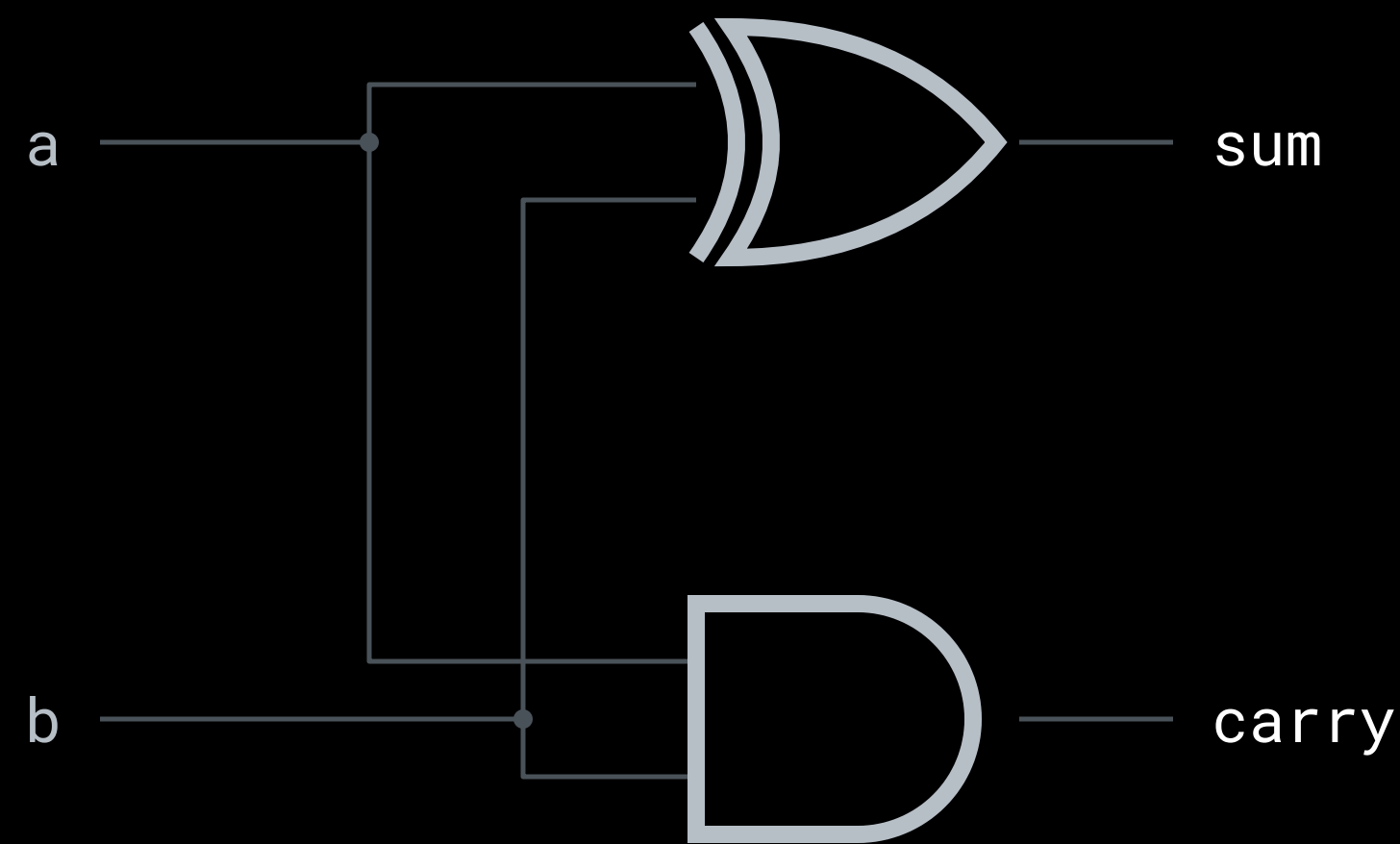
a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

the half adder: two gates



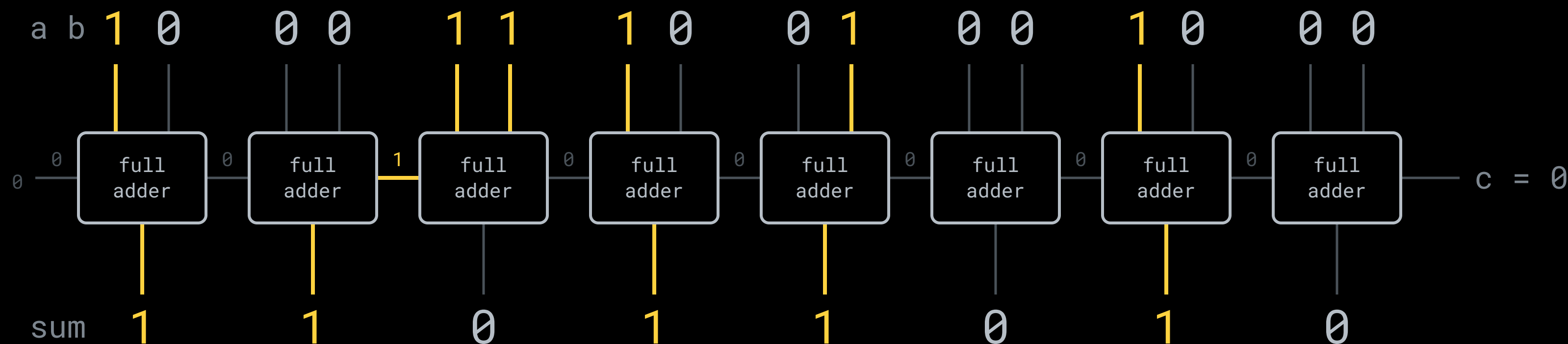
a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

the half adder: two gates



a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

eight of them in a row: a byte adder



$$\begin{array}{r} 1\ 0\ 1\ 1\ 0\ 0\ 1\ 0 = 178 \\ 0\ 0\ 1\ 0\ 1\ 0\ 0\ 0 = 40 \\ \hline 1\ 1\ 0\ 1\ 1\ 0\ 1\ 0 = 218 \end{array}$$

the carry runs from right to left, like in school.

the rest of arithmetic is built the same way

subtract = add the negative

the negative: flip every bit, then add one

multiply = add again and again

$\times 2$ is a shift to the left

compare = subtract, look at the sign

divide = subtract again and again

everything ends at the adder. the adder ends at four gates.

part 4

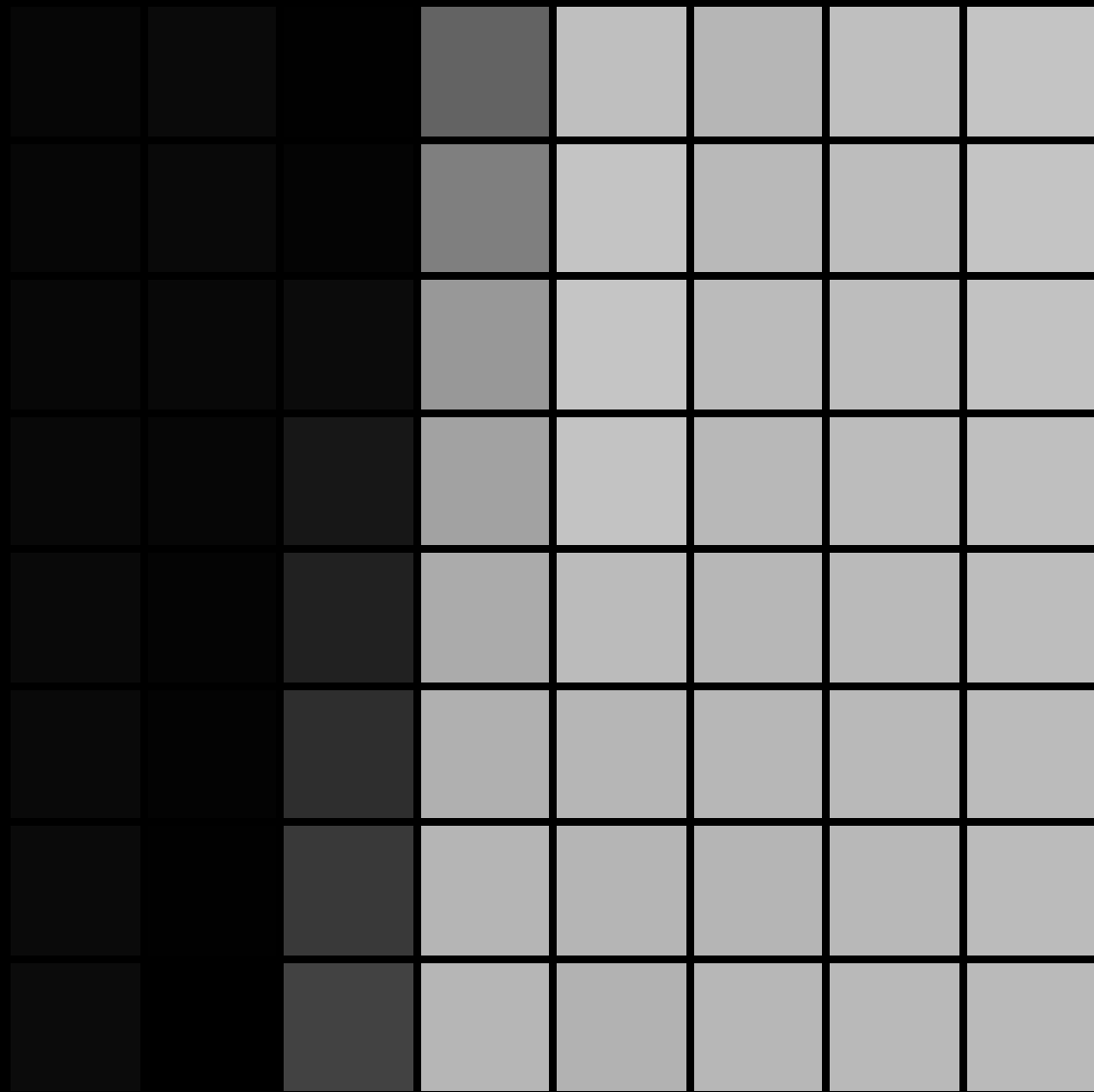
a photo, pixel by pixel

a photo is a grid of bytes

8 × 8 pixels from the parrot's beak

each pixel is one byte: 0 is black, 255 is white.

a photo is a grid of bytes



8 × 8 pixels from the parrot's beak

each pixel is one byte: 0 is black, 255 is white.

the numbers are the picture. the computer sees nothing else.

brighter: add 40 to every pixel

the yellow pixel: 178

old	1	0	1	1	0	0	1	0	178
+ 40	0	0	1	0	1	0	0	0	40
			1						
new	1	1	0	1	1	0	1	0	218

the same addition as in the byte adder

...and the other 63, then all 16,384

6	10	1	99	191	182	191	196
6	9	4	127	196	185	189	196
7	8	11	152	197	187	189	194
8	6	23	162	195	184	188	191
9	4	33	171	187	183	186	189
9	3	46	176	182	183	185	187
10	1	57	181	181	181	184	187
11	0	66	182	178	183	185	186

before

46	50	41	139	231	222	231	236
46	49	44	167	236	225	229	236
47	48	51	192	237	227	229	234
48	46	63	202	235	224	228	231
49	44	73	211	227	223	226	229
49	43	86	216	222	223	225	227
50	41	97	221	221	221	224	227
51	40	106	222	218	223	225	226

after: every pixel + 40

more filters, same toolbox

darker

`new = old - 40`

negative

`new = 255 - old`

black & white

`new = old ≥ 128 ? 255 : 0`

blend two photos

`new = (a + b) ÷ 2`

blur

`nine pixels at a time`

try it: the pixel filter

$$\text{new} = \text{old} + 40$$

add 40 to every byte, for red, green and blue separately

before: 16,384 pixels × 3 bytes



after (nothing computed yet)



step: one pixel

run: all the rest

reset

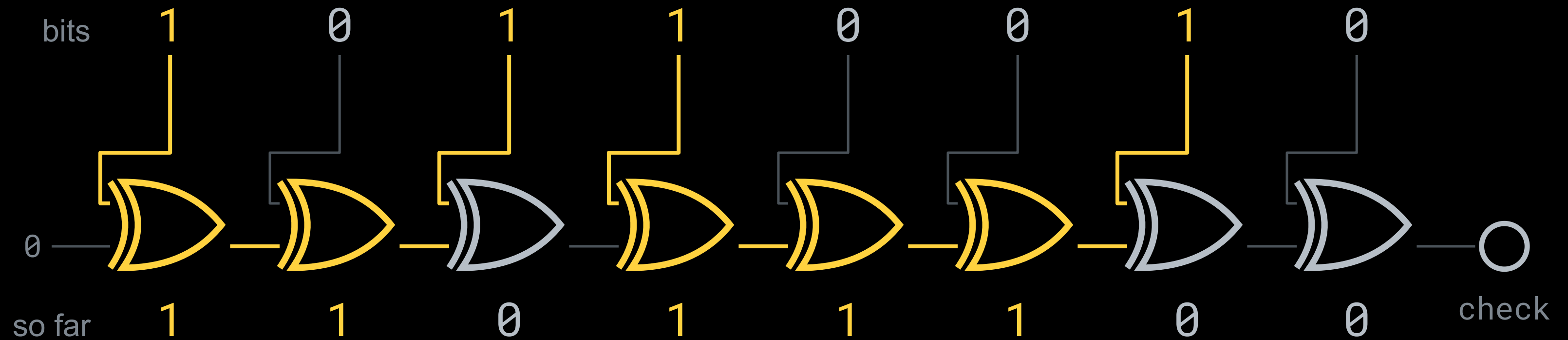
if a result leaves 0...255, the filter compares first and caps it there (clamp), like your photo app. an adder alone would drop the ninth bit: 230 + 40 would become 14.

step computes one pixel and shows the arithmetic: its bytes, bit by bit, with the carries.

part 5

one gate, two jobs

xor over a byte: a check bit



each xor takes the result so far and the next bit. four ones: even, check bit 0.

flip one bit on the way, and the check bit no longer matches.

xor with a key: encryption

message	0	1	1	0	1	0	0	1
xor key	1	1	0	0	1	0	1	0
secret	1	0	1	0	0	0	1	1

xor with a key: encryption

message	0	1	1	0	1	0	0	1
xor key	1	1	0	0	1	0	1	0
secret	1	0	1	0	0	0	1	1
secret	1	0	1	0	0	0	1	1
xor key	1	1	0	0	1	0	1	0
message	0	1	1	0	1	0	0	1

four gates, a few billion
times a second.
that is all.

